UNIVERSIDAD AUTÓNOMA JUAN MISAEL SARACHO

FACULTAD DE RECURSOS NATURALES Y TECNOLOGÍA

CARRERA DE INGENIERÍA INFORMÁTICA

MATERIA: TEL420 - SISTEMAS PARALELOS (IIY)

SEGUNDA PRÁCTICA:

DESPLIEGUE DE ARQUITECTURA Y AGENTES DE IA

Sistema de Biblioteca — Arquitectura SysLab 2.0

DOCENTE: ING. ELIAS CASSAL BALDIVIEZO

Estudiante:

Lizzy Leyla Baron Barrero

Yacuiba, Tarija - Bolivia

2 de agosto de 2026

2. Enlace al Repositorio

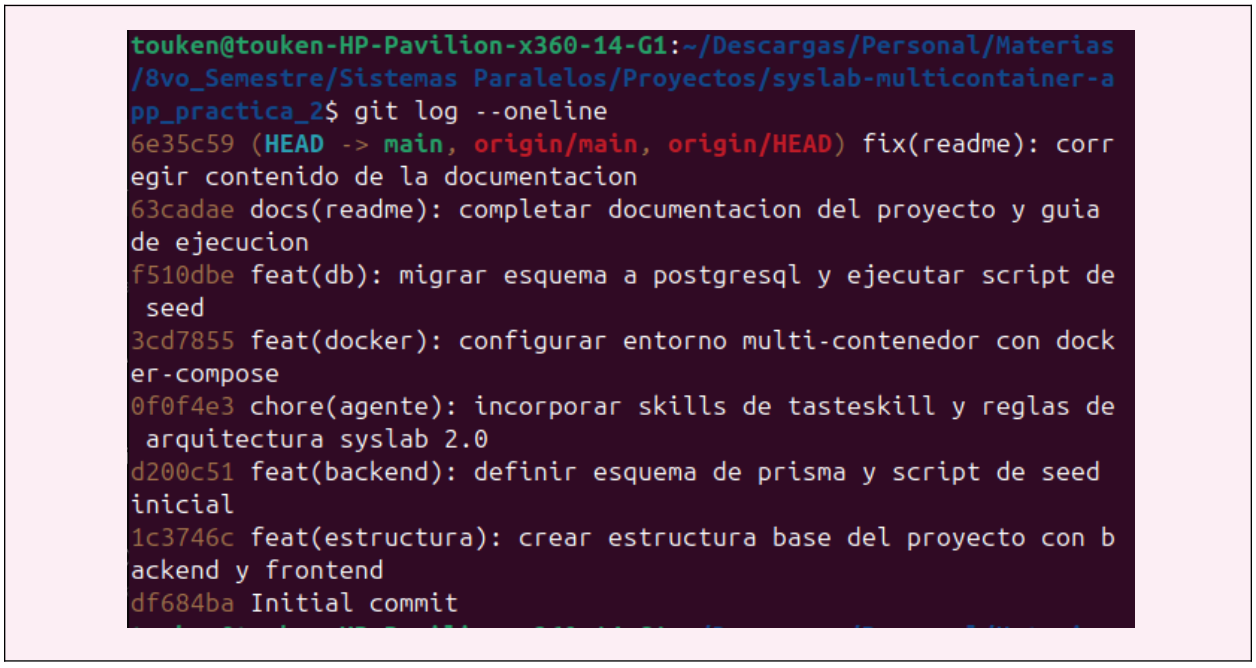
El código fuente completo del proyecto, incluyendo el historial de commits siguiendo la convención Conventional Commits, se encuentra disponible públicamente en el siguiente repositorio de GitHub:

https://github.com/Touken-byte/syslab-multicontainer-app_practica_2

El repositorio es de acceso público y contiene la totalidad de los archivos de configuración, código de backend y frontend, esquema de base de datos, y la carpeta del agente de IA con sus respectivas skills y reglas.

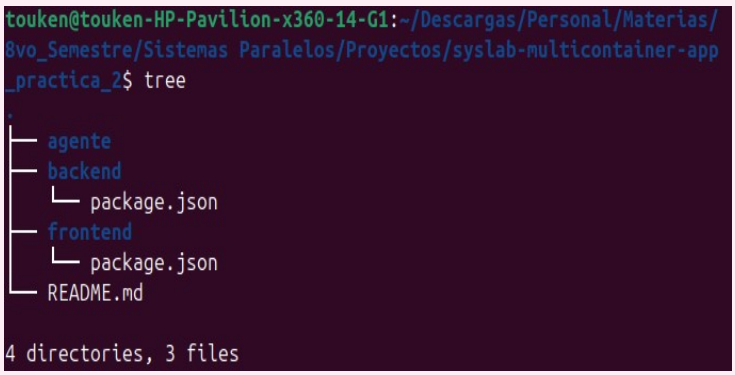
3. Historial de Commits

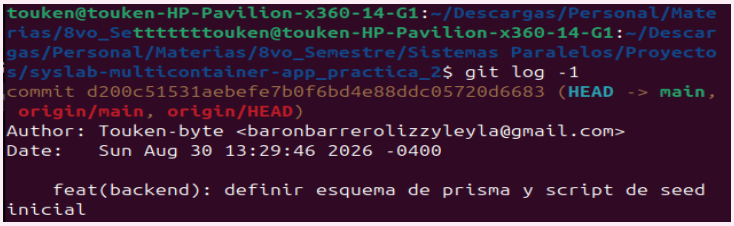
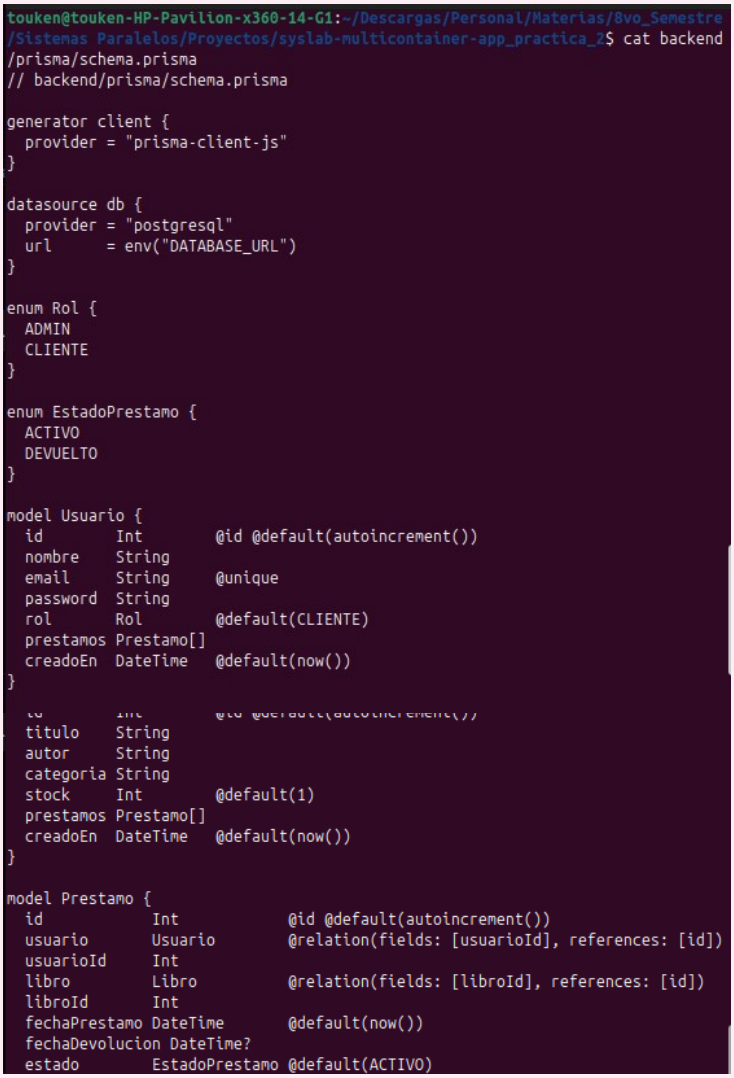
A continuación se presenta la evidencia del historial de commits del repositorio, obtenida mediante el comando git log --oneline, o bien mediante la sección "Commits" del repositorio en GitHub. Se evidencia el cumplimiento del requisito mínimo de 3 commits estructurados bajo la convención Conventional Commits, en idioma español.

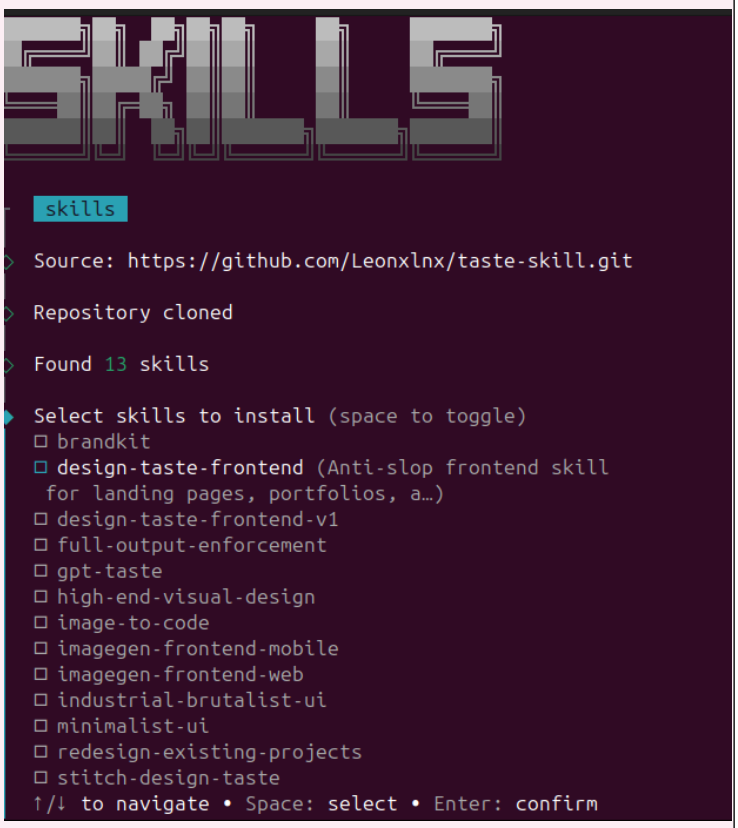
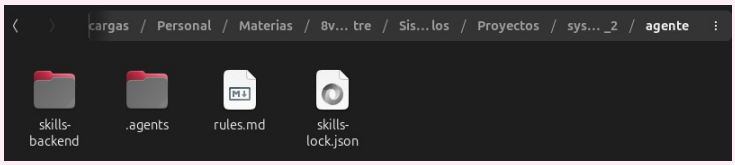
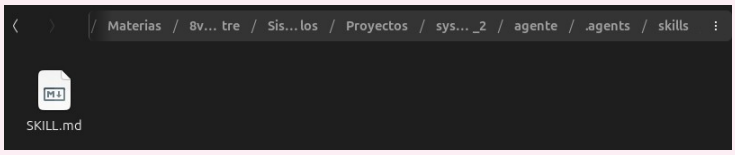
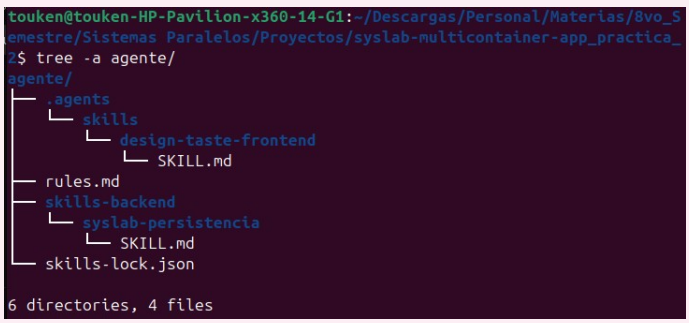


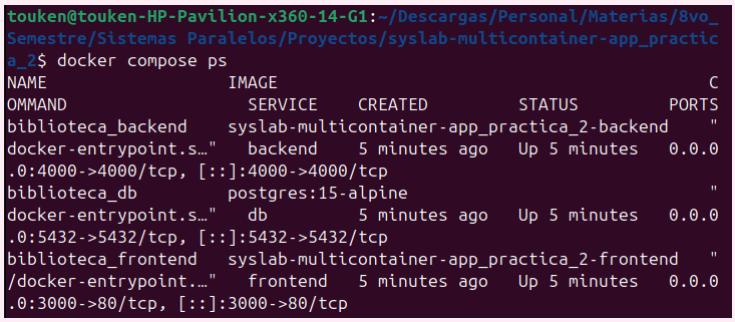
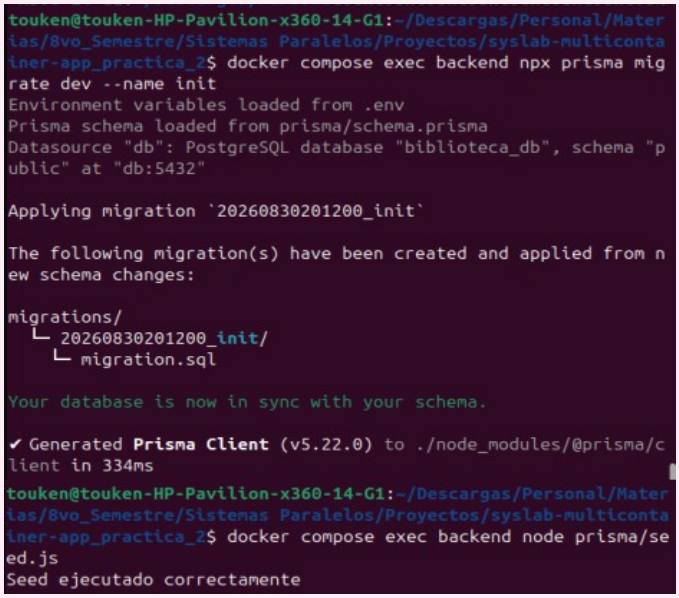
4. Evidencias del Desarrollo

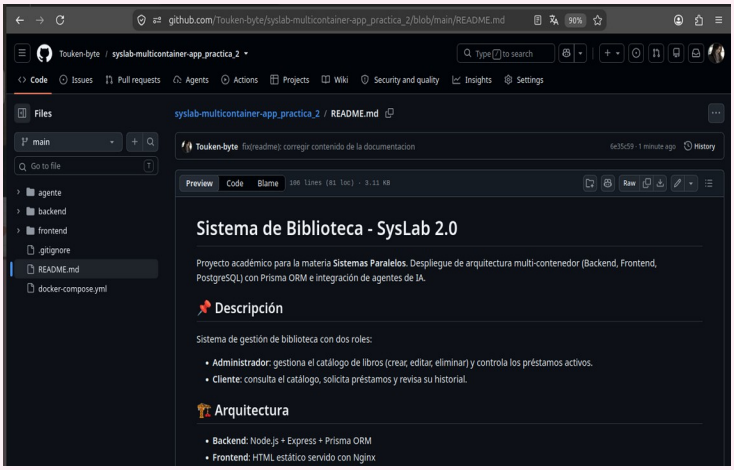
En la siguiente tabla se presentan, en orden, las seis evidencias fotográficas solicitadas en la guía práctica, junto con la explicación técnica de cada requisito. El espacio en blanco de la tercera columna está reservado para insertar la captura de pantalla correspondiente.

Requisito	Explicación Técnica	Evidencia (Captura)
Evidencia 1 — Estructura de Directorios (mkdir, npm init -y, tree)	Se creó la estructura base del proyecto con las carpetas backend, frontend y agente, y se inicializó el archivo package.json en backend y frontend mediante npm init -y. El comando tree permite visualizar de forma jerárquica la organización de carpetas y confirmar la correcta creación de los archivos de configuración iniciales.	

Requisito	Explicación Técnica	Evidencia (Captura)
Evidencia 2 — Esquema de Prisma (schema.prisma y git log -1)	Se definió el archivo schema.prisma especificando PostgreSQL como proveedor de base de datos y los modelos de datos del sistema (Usuario, Libro y Prestamo), junto con el script seed.js para la carga de datos iniciales. El comando git log -1 certifica que el cambio quedó registrado con un commit formal bajo la convención feat.	 

Requisito	Explicación Técnica	Evidencia (Captura)
Evidencia 3 — Configuración del Agente y Skills (exploración de agente/)	Se instalaron las skills base de TasteSkill dentro de agente/.agents/skills/, se crearon subcarpetas de skills personalizadas alineadas a la arquitectura SysLab 2.0 (skills-backend/syslab-persistencia) y se redactó el archivo rules.md con las reglas que gobiernan el comportamiento del backend y la persistencia de datos.	   

Requisito	Explicación Técnica	Evidencia (Captura)
Evidencia 4 — Orquestación con Docker Compose (docker compose ps)	Se crearon los archivos Dockerfile para backend y frontend, junto con el archivo docker-compose.yml en la raíz del proyecto para coordinar los tres servicios. El comando docker compose ps confirma que los tres contenedores (backend, frontend y db) se encuentran activos en estado Up.	
Evidencia 5 — Migraciones y Seed en PostgreSQL (prisma migrate dev, seed.js)	Dentro del contenedor de backend se ejecutó npx prisma migrate dev --name init para crear las tablas en PostgreSQL a partir del esquema definido, y posteriormente node prisma/seed.js para insertar los datos iniciales (usuarios, libros y un préstamo de ejemplo) en la base de datos.	

Requisito	Explicación Técnica	Evidencia (Captura)
Evidencia 6 — Documentación Final y Publicación (README.md en GitHub)	Se completó el archivo README.md en la raíz del proyecto con la descripción del sistema, arquitectura, instrucciones de ejecución y estructura de carpetas, y se publicaron todos los cambios locales al repositorio remoto de GitHub mediante git push origin main.	

5. Dificultades y Soluciones

Durante el desarrollo de la práctica surgieron distintos inconvenientes técnicos, principalmente relacionados con la compatibilidad de versiones entre Node.js y Prisma dentro del entorno de contenedores. A continuación se detallan los problemas encontrados y la solución aplicada en cada caso.

Problema / Dificultad	**Solución Aplicada**
Error de rutas locales por typo en terminal ("cd ..}") que impidió inicializar correctamente la carpeta frontend.	Se identificó el error de tipeo, se reinició el proceso desde la carpeta raíz del proyecto y se ejecutaron los comandos cd y npm init -y en pasos separados para evitar concatenaciones incorrectas.
Docker Compose no encontraba el archivo docker-compose.yml ni el .env del backend.	Se verificó con pwd que los comandos se estaban ejecutando desde una carpeta distinta a la raíz del proyecto; se corrigió la ubicación de trabajo antes de reintentar docker compose up.
Fallo en la construcción del contenedor backend: Prisma 7 requiere Node.js 20+ y la imagen base usaba node:18-alpine.	Se actualizó la imagen base del Dockerfile de backend a node:20-alpine para cumplir con los requisitos mínimos de Prisma.
Error de validación de esquema: la propiedad url dentro de datasource dejó de ser compatible con Prisma 7.	Se fijó una versión estable de Prisma y @prisma/client (5.22.0) en el package.json del backend, compatible con la sintaxis clásica de configuración usada en el proyecto.
Prisma no podía detectar la versión de OpenSSL dentro del contenedor Alpine, generando un error de parseo del motor de esquema.	Se agregó la instalación de OpenSSL en el Dockerfile del backend (RUN apk add --no-cache openssl) antes de generar el cliente de Prisma, y se reconstruyó la imagen sin caché.

6. Comandos Utilizados

La siguiente tabla resume los principales comandos empleados durante el desarrollo de la práctica y su función dentro del flujo de trabajo.

Comando	Para qué sirve
git clone <url>	Clona el repositorio remoto de GitHub en el equipo local, conectándolo automáticamente al origin.
mkdir backend frontend agente	Crea la estructura base de carpetas del proyecto.
npm init -y	Inicializa un archivo package.json con valores por defecto en backend y frontend.
mkdir -p prisma/	Crea la carpeta donde se aloja el esquema y las migraciones de Prisma.
npx prisma generate	Genera el cliente de Prisma a partir del schema.prisma para ser usado por la aplicación.
npx skills add Leonxlnx/taste-skill	Descarga e instala las skills del repositorio TasteSkill para el agente de IA.
docker compose up --build -d	Construye las imágenes y levanta los contenedores (backend, frontend, db) en segundo plano.
docker compose ps	Muestra el estado actual de los contenedores definidos en docker-compose.yml.
docker compose exec backend npx prisma migrate dev --name init	Ejecuta dentro del contenedor de backend la migración que crea las tablas en PostgreSQL según el esquema.
docker compose exec backend node prisma/seed.js	Ejecuta el script de poblado de datos iniciales dentro del contenedor de backend.
git add / git commit -m "..."	Registra los cambios en el área de preparación y crea un commit siguiendo la convención Conventional Commits.
git push origin main	Publica los commits locales hacia la rama principal del

Comando	Para qué sirve
	repositorio remoto en GitHub.

7. Conclusiones

El desarrollo de esta práctica permitió comprender de manera aplicada la arquitectura SysLab 2.0 mediante la construcción de un sistema real de biblioteca, compuesto por un backend en Node.js/Express, un frontend estático y una base de datos PostgreSQL, todos orquestados a través de Docker Compose. Este enfoque multi-contenedor evidenció la importancia de aislar responsabilidades entre servicios, así como la necesidad de gestionar correctamente las variables de entorno (.env) para cada componente del sistema, garantizando portabilidad y seguridad en la configuración.

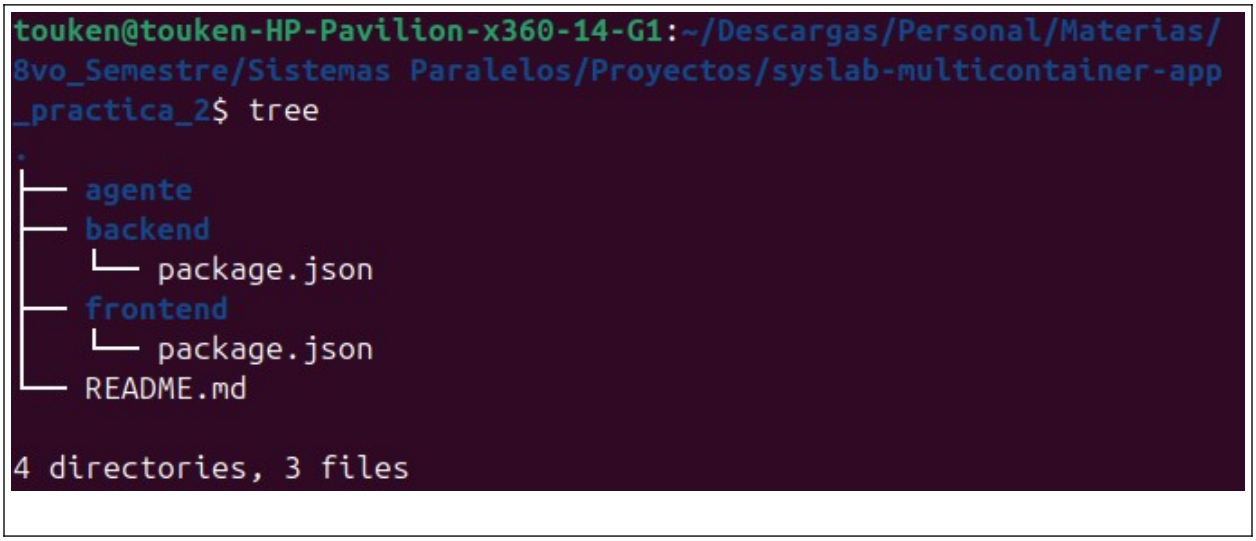
En cuanto a la gestión de persistencia, el uso de Prisma ORM demostró ser una herramienta eficiente para modelar entidades (Usuario, Libro y Préstamo) y sus relaciones de forma declarativa, además de automatizar la generación de migraciones sobre PostgreSQL. Durante la implementación se evidenció la importancia de mantener coherencia entre las versiones de Node.js y de Prisma, dado que versiones recientes del ORM (v7) modificaron su forma de configurar la conexión a la base de datos, lo cual exigió fijar versiones estables para asegurar la compatibilidad dentro del entorno de contenedores basados en Alpine Linux.

Respecto a la integración de agentes de IA, la incorporación de las skills de TasteSkill junto con las skills personalizadas y el archivo de reglas (rules.md) permitió establecer lineamientos claros sobre cómo un agente debe comportarse al generar o modificar código relacionado con la persistencia y la arquitectura del backend. Esto refuerza la idea de que los agentes de IA, para ser

útiles en un flujo de desarrollo profesional, requieren de un marco de reglas y buenas prácticas explícitas, en lugar de operar de manera genérica.

Finalmente, el uso de la convención Conventional Commits a lo largo de todo el proceso permitió mantener un historial de versiones ordenado, trazable y fácil de auditar, lo cual constituye una práctica fundamental en el trabajo colaborativo de desarrollo de software. En conjunto, esta práctica fortaleció competencias relacionadas con la containerización de aplicaciones, la administración de bases de datos relacionales mediante ORM, y la incorporación estructurada de agentes de inteligencia artificial dentro de un flujo de trabajo de ingeniería de software.

8. Anexos



```
touken@touken-HP-Pavilion-x360-14-G1:~/Descargas/Personal/Materias/
8vo_Semestre/Sistemas Paralelos/Proyectos/syslab-multicontainer-app
_practica_2$ tree
.
├── agente
├── backend
│   └── package.json
├── frontend
│   └── package.json
└── README.md

4 directories, 3 files
```

```
touken@touken-HP-Pavilion-x360-14-G1:~/Descargas/Personal/Materias/8vo_Semestre
/Sistemas Paralelos/Proyectos/syslab-multicontainer-app_practica_2$ cat backend
/prisma/schema.prisma
// backend/prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum Rol {
  ADMIN
  CLIENTE
}

enum EstadoPrestamo {
  ACTIVO
  DEVUELTO
}

model Usuario {
  id        Int        @id @default(autoincrement())
  nombre    String
  email     String     @unique
  password  String
  rol       Rol        @default(CLIENTE)
  prestamos Prestamo[]
  creadoEn  DateTime   @default(now())
}
```

```
model Libro {
  id        Int        @id @default(autoincrement())
  titulo    String
  autor     String
  categoria String
  stock     Int        @default(1)
  prestamos Prestamo[]
  creadoEn  DateTime   @default(now())
}

model Prestamo {
  id            Int            @id @default(autoincrement())
  usuario       Usuario        @relation(fields: [usuarioId], references: [id])
  usuarioId     Int
  libro         Libro          @relation(fields: [libroId], references: [id])
  libroId       Int
  fechaPrestamo DateTime       @default(now())
  fechaDevolucion DateTime?
  estado        EstadoPrestamo @default(ACTIVO)
```

```
touken@touken-HP-Pavilion-x360-14-G1:~/Descargas/Personal/Materias/8vo_Setttttttouken@touken-HP-Pavilion-x360-14-G1:~/Descargas/Personal/Materias/8vo_Semestre/Sistemas Paralelos/Proyectos/syslab-multicontainer-app_practica_2$ git log -1
commit d200c51531aebefe7b0f6bd4e88ddc05720d6683 (HEAD -> main, origin/main, origin/HEAD)
Author: Touken-byte <baronbarrerolizzyleyla@gmail.com>
Date:   Sun Aug 30 13:29:46 2026 -0400

    feat(backend): definir esquema de prisma y script de seed inicial
```

```
 ███████╗██╗  ██╗██╗██╗     ██╗     ███████╗
 ██╔════╝██║ ██╔╝██║██║     ██║     ██╔════╝
 ███████╗█████╔╝ ██║██║     ██║     ███████╗
 ╚════██║██╔═██╗ ██║██║     ██║     ╚════██║
 ███████║██║  ██╗██║███████╗███████╗███████║
 ╚══════╝╚═╝  ╚═╝╚═╝╚══════╝╚══════╝╚══════╝

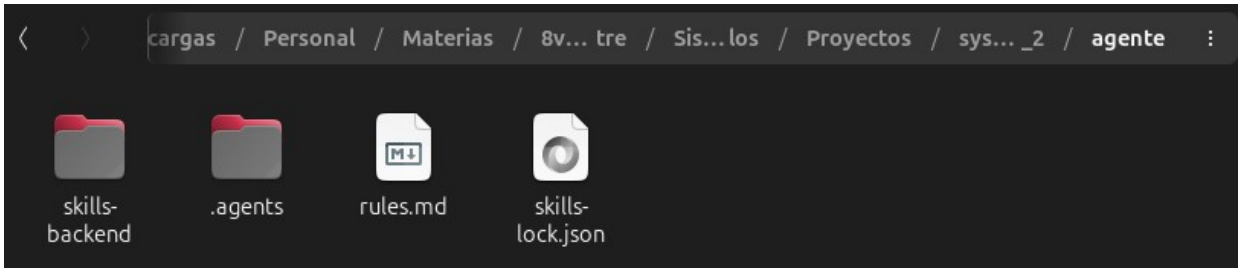
  skills

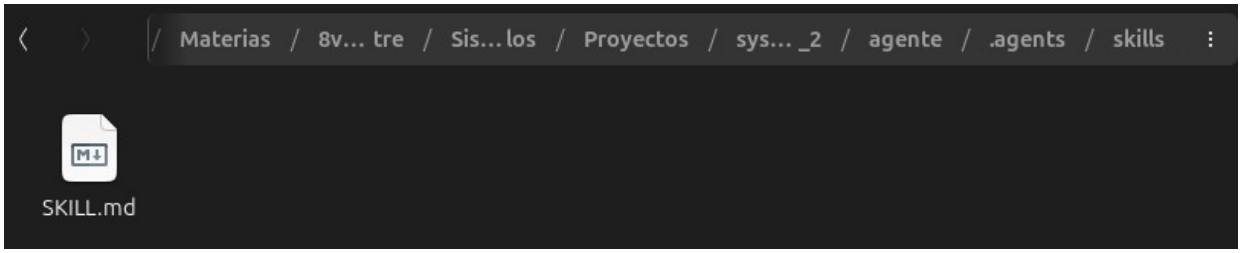
◇ Source: https://github.com/Leonxlnx/taste-skill.git

◇ Repository cloned

◇ Found 13 skills

◆ Select skills to install (space to toggle)
  ☐ brandkit
  ☐ design-taste-frontend (Anti-slop frontend skill
    for landing pages, portfolios, a…)
  ☐ design-taste-frontend-v1
  ☐ full-output-enforcement
  ☐ gpt-taste
  ☐ high-end-visual-design
  ☐ image-to-code
  ☐ imagegen-frontend-mobile
  ☐ imagegen-frontend-web
  ☐ industrial-brutalist-ui
  ☐ minimalist-ui
  ☐ redesign-existing-projects
  ☐ stitch-design-taste
  ↑/↓ to navigate • Space: select • Enter: confirm
```






```
touken@touken-HP-Pavilion-x360-14-G1:~/Descargas/Personal/Materias/8vo_Semestre/Sistemas Paralelos/Proyectos/syslab-multicontainer-app_practica_2$ tree -a agente/
agente/
├── .agents
│   └── skills
│       └── design-taste-frontend
│           └── SKILL.md
├── rules.md
├── skills-backend
│   └── syslab-persistencia
│       └── SKILL.md
└── skills-lock.json

6 directories, 4 files
```

```
touken@touken-HP-Pavilion-x360-14-G1:~/Descargas/Personal/Materias/8vo_Semestre/Sistemas Paralelos/Proyectos/syslab-multicontainer-app_practica_2$ docker compose ps
NAME                   IMAGE                                                C
OMMAND                   SERVICE    CREATED         STATUS         PORTS
biblioteca_backend     syslab-multicontainer-app_practica_2-backend     "
docker-entrypoint.s…"    backend    5 minutes ago   Up 5 minutes   0.0.0
.0:4000->4000/tcp, [::]:4000->4000/tcp
biblioteca_db          postgres:15-alpine                               "
docker-entrypoint.s…"    db         5 minutes ago   Up 5 minutes   0.0.0
.0:5432->5432/tcp, [::]:5432->5432/tcp
biblioteca_frontend    syslab-multicontainer-app_practica_2-frontend    "
/docker-entrypoint.…"    frontend   5 minutes ago   Up 5 minutes   0.0.0
.0:3000->80/tcp, [::]:3000->80/tcp
```

```
-x360-14-G1:~/Descargas/Personatouktouktouktouktouktouktouktouk
touken@touken-HP-Pavilion-x360-14-G1:~/Descargas/Personal/Mater
ias/8vo_Semestre/Sistemas Paralelos/Proyectos/syslab-multiconta
iner-app_practica_2$ docker compose exec backend npx prisma mig
rate dev --name init
Environment variables loaded from .env
Prisma schema loaded from prisma/schema.prisma
Datasource "db": PostgreSQL database "biblioteca_db", schema "p
ublic" at "db:5432"

Applying migration `20260830201200_init`

The following migration(s) have been created and applied from n
ew schema changes:

migrations/
  └─ 20260830201200_init/
    └─ migration.sql

Your database is now in sync with your schema.

✔ Generated Prisma Client (v5.22.0) to ./node_modules/@prisma/c
lient in 334ms
```

```
touken@touken-HP-Pavilion-x360-14-G1:~/Descargas/Personal/Mater
ias/8vo_Semestre/Sistemas Paralelos/Proyectos/syslab-multiconta
iner-app_practica_2$ docker compose exec backend node prisma/se
ed.js
Seed ejecutado correctamente
```



Lizzy Leyla Baron Barrero uwu